Optimisation, Application Connection & Capstone

Module 8 • Measure, connect and deliver one reliable order workflow.

MODULE

08

developer_lab finale

Module 8 connects database evidence to application behaviour

1

MEASURE

Capture a plan and identify where rows, buffers and time are spent.

2

IMPROVE

Change the smallest justified part: query, index, statistics or data access.

3

DELIVER

Run the final order workflow through safe Python transactions.

Performance and correctness are both observable outcomes—not assumptions.

By the end, learners can complete seven tasks

01 Read estimated and actual execution-plan values

02 Recognise scans, joins, sorts and expensive loops

03 Refresh statistics and justify an index or rewrite

04 Connect with Psycopg and bind values safely

05 Control commit, rollback, timeouts and errors

06 Build the transactional order-processing capstone

07 **Prove success and failure with repeatable tests**

SUCCESS = VALID ORDERS COMMIT; INVALID ORDERS LEAVE NO TRACE

Optimisation is a loop, not a one-time command

1 BASELINE

Choose a representative query and record its inputs, output grain and elapsed behaviour.

2 EXPLAIN

Compare estimates with actual rows, loops, buffers, sorts and scan choices.

3 CHANGE + RETEST

Apply one justified change, then rerun under comparable conditions.

Keep the change only when the evidence improves the real workload.

Start with the workload and its success measure

QUERY CONTEXT

Report: completed order value by customer
Filter: recent date range
Sort: highest value first
Expected grain: one row per customer
Representative data: realistic distribution

MEASURE

Correct result and row count
Planning and execution time
Rows removed by filters
Buffer hits and reads
Repeatability across parameters

“Fast” is incomplete unless the query, data and success threshold are named.

EXPLAIN predicts; ANALYZE executes and measures

```
EXPLAIN (COSTS, VERBOSE)
SELECT ...;

EXPLAIN (ANALYZE, BUFFERS, TIMING OFF)
SELECT ...;

BEGIN;
EXPLAIN (ANALYZE, BUFFERS) UPDATE ...;
ROLLBACK;
```

ESTIMATED PLAN

Safe for inspection; no query execution statistics.

ACTUAL PLAN

Runs the statement. Wrap data-changing tests in a transaction and roll back.

Never run EXPLAIN ANALYZE on a risky write without controlling its side effects.

Read a plan from the deepest node upward

```
Sort  (actual rows=84 loops=1)
  Sort Key: (sum(...)) DESC
  -> HashAggregate  (actual rows=84 loops=1)
       Group Key: c.customer_id
       -> Hash Join  (actual rows=1260 loops=1)
            Hash Cond: (o.customer_id = c.customer_id)
            -> Seq Scan on orders o
                 Filter: (status = 'COMPLETED')
```

NODE

What operation occurred?

FLOW

How many rows entered and left?

COST

Where did loops, time or buffers grow?

A scan choice is correct only in context

PLAN NODE	LIKELY FIT	TRADE-OFF
Sequential scan	Large share of table	Reads table pages directly
Index scan	Selective lookup	Index order plus heap access
Index-only scan	Covered and visible rows	May avoid heap reads
Bitmap scan	Moderate result set	Collects locations before heap access

A sequential scan is not automatically a problem; selectivity decides.

Design indexes around predicates, joins and order

```
CREATE INDEX orders_customer_created_idx
ON app.orders (customer_id, created_at DESC)
INCLUDE (status);

CREATE INDEX orders_open_created_idx
ON app.orders (created_at DESC)
WHERE status = 'OPEN';
```

COLUMN ORDER

Match leading equality conditions before range or ordering needs.

PARTIAL / INCLUDE

Reduce index scope or cover returned columns when the workload justifies it.

Each index adds storage and write maintenance—prove that it earns its cost.

Statistics shape the planner’s decisions

```
ANALYZE app.orders;

SELECT attname, n_distinct, most_common_vals
FROM pg_stats
WHERE schemaname = 'app'
  AND tablename = 'orders';

ALTER TABLE app.orders
  ALTER COLUMN status SET STATISTICS 500;
ANALYZE app.orders(status);
```

AFTER BULK LOAD

Refresh statistics before judging the new plan.

WHEN SKEW MATTERS

Increase a column target only when better estimates justify the extra analysis work.

Statistics describe data distribution; indexes provide access paths.

Query shape often matters more than a new index

FILTER EARLY

Restrict rows with sargable predicates; avoid wrapping indexed columns unnecessarily.

REMOVE WORK

Return only needed columns; avoid repeated subqueries and unneeded DISTINCT operations.

MATCH THE GRAIN

Aggregate once at the business grain and join only the rows required.

Optimise semantics first: the rewritten query must return the same correct answer.

Demonstration: improve one customer-value report

```
EXPLAIN (ANALYZE, BUFFERS, TIMING OFF)
SELECT c.customer_id, c.full_name,
       SUM(i.quantity * i.unit_price) AS value
FROM app.customers c
JOIN app.orders o USING (customer_id)
JOIN app.order_items i USING (order_id)
WHERE o.status = 'COMPLETED'
  AND o.created_at >= CURRENT_DATE - INTERVAL '90 days'
GROUP BY c.customer_id, c.full_name
ORDER BY value DESC;
```

1 CAPTURE

Save the original plan and row count.

2 CHANGE

Refresh statistics; test one index or rewrite.

3 COMPARE

Check plan, buffers and identical results.

Production tuning starts with repeated workload evidence

```
SELECT query, calls, total_exec_time, mean_exec_time, rows
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 10;

SELECT pid, state, wait_event_type, wait_event, query
FROM pg_stat_activity
WHERE datname = current_database();
```

pg_stat_statements

Ranks normalised statements by accumulated planning and execution behaviour.

pg_stat_activity

Shows current sessions, waits and active SQL—a live snapshot, not history.

Measure frequent expensive work, not only the slow query someone noticed once.

The application owns workflow; PostgreSQL owns data integrity

PYTHON SERVICE

Validate request shape
Open database unit of work
Map results to the response

PSYCOPG

Bind Python values
Manage connection and cursor
Expose PostgreSQL errors

POSTGRESQL

Enforce constraints
Lock and update rows
Commit or roll back atomically

Keep business invariants enforced in the database even when the application validates first.

Connect with configuration outside the source code

```
python -m pip install "psycopg[binary]"

# PowerShell
$env:DATABASE_URL = "postgresql://app_user:***@localhost:5432/developer_lab"

import os, psycopg

with psycopg.connect(os.environ["DATABASE_URL"]) as conn:
    row = conn.execute("SELECT current_database(), current_user").fetchone()
    print(row)
```

DO

Use a restricted application role and environment/secret configuration.

AVOID

Hard-coded passwords
Superuser application access
Logging complete DSNs
Unbounded connection attempts

Bind values; compose identifiers separately

```
# Values use %s placeholders
row = conn.execute(
    "SELECT product_id, price FROM app.products WHERE sku = %s",
    (sku,),
).fetchone()

# Do not quote the placeholder
# Do not build SQL with f-strings or + user input
```

VALUES

Parameters preserve types and prevent input from becoming SQL structure.

IDENTIFIERS

For dynamic table or column names, use psycopg.sql.Identifier—or avoid dynamic structure.

A single value still needs a one-element tuple: (sku,).

Transaction scope should match one business outcome

```
with psycopg.connect(DSN, autocommit=True) as conn:
    with conn.transaction():
        order_id = create_order(conn, customer_id)
        for item in items:
            reserve_stock(conn, item.sku, item.qty)
            add_order_item(conn, order_id, item)
        total = calculate_total(conn, order_id)
# COMMIT on success; ROLLBACK if an exception escapes
```

KEEP IT SHORT

Do not wait for user input or external services while database locks are held.

AFTER FAILURE

A failed transaction cannot continue normally; roll back before reusing the connection.

Pools, timeouts and observability protect shared capacity

POOL

Reuse a bounded number of healthy connections; return each one promptly.

TIMEOUT

Set connect and statement limits appropriate to the request path.

OBSERVE

Tag application_name; record duration, outcome and safe correlation IDs.

A pool manages scarce sessions; it does not make long transactions harmless.

Demonstration: one repository function, one safe query

```
from psycopg.rows import dict_row

def find_products(conn, maximum):
    sql = """
      SELECT sku, name, price, stock_qty
      FROM app.products
      WHERE price <= %s AND stock_qty > 0
      ORDER BY price, sku
    """
    return conn.execute(sql, (maximum,)).fetchall()

with psycopg.connect(DSN, row_factory=dict_row) as conn:
    for product in find_products(conn, 250):
        print(product["sku"], product["price"])
```

CHECK

Correct types
Expected ordering
Empty-result behaviour

OBSERVE

Log duration safely
Run EXPLAIN in psql
Verify the result uses the same predicate

Capstone: place an order without partial state

INPUT

Customer email
SKU and positive quantity pairs

TRANSACTION

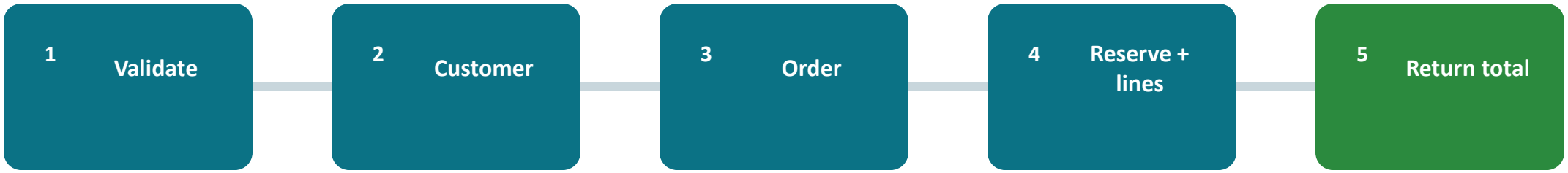
Find or create customer
Create order
Reserve stock
Insert line items

OUTPUT

order_id
calculated total
or actionable error

Acceptance: valid input commits; unknown SKU, low stock or invalid quantity leaves no partial order.

The capstone transaction has one commit boundary



ANY ERROR

Exception escapes the transaction block → PostgreSQL rolls back every step.

ALL STEPS SUCCEED

Context exits normally → one commit makes the order visible.

Do not commit inside helper functions; the service-level workflow owns the boundary.

Starter implementation keeps SQL parameterised

```
def place_order(conn, email, items):
    with conn.transaction():
        customer_id = get_or_create_customer(conn, email)
        order_id = conn.execute(
            "INSERT INTO app.orders(customer_id) VALUES (%s) RETURNING order_id",
            (customer_id,),
        ).fetchone()[0]

        for sku, qty in items:
            product_id, unit_price = reserve_product(conn, sku, qty)
            conn.execute(
                "INSERT INTO app.order_items(order_id, product_id, quantity, unit_price) "
                "VALUES (%s, %s, %s, %s)",
                (order_id, product_id, qty, unit_price),
            )
        return order_id, order_total(conn, order_id)
```

Complete the helper functions, error mapping and input checks—then prove rollback.

Atomic stock reservation prevents overselling

```
UPDATE app.products
SET stock_qty = stock_qty - %s
WHERE sku = %s
  AND stock_qty >= %s
RETURNING product_id, price, stock_qty;
```

ONE STATEMENT

The condition and decrement occur under one row update lock.

NO ROW RETURNED

Distinguish unknown SKU from insufficient stock with a follow-up lookup or a trusted function.

Avoid SELECT stock → check in Python → UPDATE; concurrent sessions can invalidate the gap.

Hands-on: build, test and explain the service

10 MIN	Prepare DSN, connect, verify role and inspect starting stock.
20 MIN	Implement helpers and the service-level transaction.
20 MIN	Run success and failure tests; verify database state.
10 MIN	**Capture one plan and explain one performance decision.**

Work in pairs: driver writes; navigator checks transaction boundaries and evidence.

Validation must prove both commit and rollback

TEST	DATABASE EVIDENCE	OUTCOME
Valid customer + SKU	Order and lines created; stock decreases	COMMIT
Unknown SKU	No order or lines persist; stock unchanged	ROLLBACK
Insufficient stock	No partial item or order persists	ROLLBACK
Quantity ≤ 0	Rejected before or by database constraint	NO CHANGE
Repeated request	Defined behaviour: duplicate blocked or idempotent	EXPLAIN

Record before-and-after row counts and stock values; screenshots alone are not a test.

Assessment rewards correctness, evidence and explanation

CRITERION	MARKS	EVIDENCE
Transaction correctness	**30**	All required writes share one safe boundary
SQL and parameters	**20**	Values bound safely; constraints respected
Failure handling	**20**	Useful messages; rollback verified
Performance evidence	**20**	Plan read correctly; change justified
Code quality	**10**	Clear functions, configuration and safe logs

Pass condition: no partial order remains after any tested failure.

Troubleshoot from the smallest failing boundary

SYMPTOM	LIKELY CAUSE	NEXT CHECK
connection refused	Server/port/container	Check readiness and mapped port
password authentication failed	Role or credential	Verify user, secret and pg_hba rule
current transaction is aborted	Earlier SQL error	Roll back, then fix the first error
query still scans sequentially	Low selectivity or small table	Compare rows returned and buffers
order remains after failure	Commit boundary is fragmented	Remove helper commits; retest rollback

Fix the first failing boundary; later symptoms may be consequences.

Knowledge check: defend each engineering decision

PLAN

Which node did most work, and what evidence proves it?

TRANSACTION

Where is the only commit boundary, and why is it placed there?

FAILURE

How do you prove that an invalid order leaves no partial state?

A strong answer connects the decision to observable database behaviour.

Use official references for the next implementation

POSTGRESQL 18 DOCUMENTATION

EXPLAIN and Using EXPLAIN
ANALYZE statistics
pg_stat_activity
pg_stat_statements

postgresql.org/docs/current/

PSYCOPG 3 DOCUMENTATION

Basic module usage
Passing parameters
Transaction management
Connection pools

psycopg.org/psycopg3/docs/

Recheck version-specific behaviour before production deployment.

Close the course: design, prove and observe

1

DESIGN

Use types, keys, constraints and transaction boundaries to reject invalid state.

2

PROVE

Use parameterised code and repeatable tests to demonstrate correct outcomes.

3

OBSERVE

Use plans, statistics and safe logs to explain real behaviour.

Exit ticket: show one decision you changed because the database evidence changed your mind.